



CyberChallenge.IT is the first Italian training program in cybersecurity for high-school and undergraduate students.

A horizontal banner image showing a group of students in a museum setting, looking at various ancient statues and artifacts.

Rev 2

Simone Sulis, Daniele Orrù, Davide Maiorca
Referenti: Sofia Tocco, Silvia Sanna

Università degli Studi di Cagliari



GDPR

GARANTE
PER LA PROTEZIONE
DEI DATI PERSONALI



<https://cyberchallenge.it>

<https://srdnln.it>



CYBER
CHALLENGE.IT



Symbolic Execution



Z3 is a Microsoft project to prove theorems and solve mathematical equations.

Various fields covered, such as biology and geometry problems, constraint solving, testing, security, hybrid systems' analysis, and more.

In reverse engineering tasks, Z3 is able to solve complex **constraints** (satisfy certain mathematical constraints) and extracting valid inputs.

<https://pypi.org/project/z3-solver/> □ *pip install z3-solver*





- Z3 employs variables
- **Boolean Variables** are declared using `Bool('x')`.
- **Integer Variables** are declared using `Int('x')`.
- **Real Variables** are declared using `Real('x')`.
- **Bit Vector Variables** are declared using `BitVec('x', size)` where size is the number of bits.
- Array Variables are declared using `Array('x', domain_type, range_type)`
- The mathematical conditions are given through **constraints**
- Constraints are then given to a **solver**

Z3



Instruction	Description
<code>from z3 import *</code>	Imports all the definitions from the Z3 Python library.
<code>s = Solver()</code>	Creates a new Z3 solver instance.
<code>BitVecs("x0 x1 x2 x3", 4)</code>	Defines a sequence of 4-bit vector variables.
<code>Real('x') Int('x')</code>	Defines a real int number variable x.
<code>s.add(...)</code>	Adds constraints to the solver.
<code>s.check()</code>	Checks if the given constraints are satisfiable.
<code>s.model()</code>	Retrieves the model that satisfies the constraints if one exists.
<code>z3.sat z3.unsat z3.unknown</code>	Given set of constraints or formula is satisfiable unsatisf unknown



```
# Import
from z3 import *

# Define the variable
x = Real('x') # x is a variable of 'Real' type
y = Real('y') # y is a variable of 'Real' type

# Define the solver
s = Solver() # s is the object for the solution

# All needed operations
s.add(x+y > 5, x > 1, y > 1)

# Get the result
if (s.check()): # Check if the solver assertions and assumptions are consistent
    print(s.model()) # Returns the solved model
```



```
from z3 import *

s = Solver()

# Define BitVec variables for both side of the equation (as output variables and as arguments)
x0, x1, x2, x3 = BitVecs("x0 x1 x2 x3", 4)

# Adding a constraint
s.add((0+x1+((x0^x1)) == 296))

# Checks if the problem is solvable
print(s.check())
model = s.model() # Save the solved model

flag = ""
for v in [x0, x1, x3, x3]: # Iterate through each variable of the solution
    flag += chr(model[v].as_long()) # Convert the numbers to characters

print(flag)
```



Angr is a Python tool designed to solve tasks in static, dynamic and symbolic analysis

Capabilities of Angr:

- Test a set of possible inputs
- Test all possible execution paths
- Discover conditions of program's state changes
- How to reach a solution





Docker enables the execution of Angr in an isolated container, simplifying the setup process, without affecting the host system's environment

Action	Command
Install using docker	<code>docker pull angr/angr</code>
Run angr via docker	<code>docker run -it angr/angr</code>
Mount directory inside docker	<code>docker run -it -v local_dir_path:/home/angr/angr-dev/work angr/angr</code>
Run angr inside docker	<code>python;</code> <code>import angr</code>
Exit docker	Inside docker: <code>exit</code>



Claripy is an abstracted constraint-solving wrapper used for:

- Handle symbolic expressions
- Simplify constraint-solving tasks in binary analysis

The general use is similar to z3.

It is integrated with Angr, automating most interactions and providing a robust backend for symbolic execution

```
import claripy
a = claripy.BVV(3, 32)
b = claripy.BVS('var_b', 32)
s = claripy.Solver()
s.add(b > a)
print(s.eval(b, 1)[0])
```



Angr – Start a project

Action	Code
Load the binary	<pre>proj = angr.Project('file.bin', load_options={'auto_load_libs':True False})</pre>
Define the initial state: you can do it with blank_state , entry_state or full_initial_state	<pre>initial_state = proj.factory.entry_state()</pre>
Create simulation manager from initial state	<pre>sim = proj.factory.simulation_manager(initial_state)</pre>
Explore the execution paths after having set goal and avoid addresses	<pre>sim.explore(find=find_addr, avoid=avoid_addr)</pre>
Check the discovered paths	<pre>if simgr.found: found = sim.found[0] stdin = found.posix.dumps(0) stdout = found.posix.dumps(1)</pre>

A [solver](#) is a powerful interface that handles symbolic expressions, applies constraints and computes possible solutions for variables in the context of an execution state

Bit Vectors

BitVector Value (BVV): Represents a concrete value

BitVector Symbol (BVS): Represents a symbolic value

Floating Points

Floating Point Value (FPV): a concrete floating-point variable

Floating Point Symbol (FPS): a symbolic floating-point variable

Action	Code
Define a BitVector	<pre>var = state.solver.BVV(insert_numer, insert_number_of_bits) -> ones = state.solver.BVV(1, 64)</pre>
Define a Floating Point	<pre>var = state.solver.FPV() var = state.solver.FPS()</pre>
Add constraints	<pre>state.solver.add(x>9)</pre>
Evaluate	<pre>state.solver.eval(x); eval_one; eval_upto; eval_atleast;</pre>
Satisfying	<pre>state.satisfiable()</pre>



A **State** object provides a representation of a program's machine state, allowing for detailed inspection and modification of registers, memory, and more.

Action	Code
Read content of a register	<code>state.regs.rsp</code>
Copy a register to another	<code>state.regs.rbp = state.regs.rsp</code>
Store rdx to memory at 0x1000	<code>state.mem[0x1000].uint64_t = state.regs.rdx</code>
Dereference rbp	<code>state.regs.rbp = state.mem[state.regs.rbp].uint64_t.resolved</code>
'add rax, qword ptr[rsp+8]'	<code>state.regs.rax += state.mem[state.regs.rsp+8].uint64_t.resolved</code>
Successors	<code>nxt = state.step() print(nxt.successors[0])</code>
Breakpoint	<code>state.inspect.b('mem_read') # Read memory in the current state</code>

Simulator:

Set the simulation manager	<code>sim = proj.factory.simgr(state)</code>
Check simulation manager status	<code>sim.active</code>
Advance the execution by one step	<code>sim.step()</code>

Stash

Collection or group of states that the SimulationManger manages during symbolic execution (organize different groups of states according to their characteristics or the result of their analysis). Different types: active, deadended, pruned, unconstrained, unsat

Action	Code
Set the simulation manager	<code>sim = proj.factory.simgr(state)</code>

Exploration

Find a state reaching specific address while discarding all states that go through another address (run execution until reaches a state matching the condition in *find* (address, list of addr, function...))

Action	Code
Execute until condition	<code>sim.explore(find=lambda s: b"Congrats" in s.posix.dumps(1)); sim.found[0].posix.dumps(0); # Retrieve the condition</code>



Instruction	Description
<code>angr.Project(binary_path)</code>	Load a binary into an Angr project for analysis.
<code>claripy.BVS(...)</code>	Create a symbolic bitvector variable with a given size.
<code>project.factory.full_init_state()</code>	Initialize a simulation state for the project.
<code>state.add_constraints(...)</code>	Apply constraints to the state to limit the possibilities during analysis.
<code>sym_variable.get_byte(...)</code>	Get a specific byte from a symbolic bitvector.
<code>proj.factory.simgr(...)</code>	Create a simulation manager for the project's state.
<code>simgr.explore(...)</code>	Direct the simulation manager to explore the binary's state space.
<code>solver.eval(...)</code>	Evaluate the symbolic expression to a concrete value.

<https://docs.angr.io/en/latest/examples.html>



```
#Importing necessary libraries for binary analysis
```

```
import angr
```

```
import claripy
```

```
#Loading the binary into angr project
```

```
proj = angr.Project(executable_name, main_opts={'base_addr':base_addr})
```

```
#Creating a symbolic variable of size 5 bytes
```

```
sym_var_size = 5
```

```
sym_var = claripy.BVS('sym_variable', 8*sym_var_size)
```

```
#Initializing a new simulation state with the symbolic variable as input
```

```
stdin = angr.SimPackets(name='stdin', content=[(sym_var, sym_var_size)])
```

```
state = proj.factory.full_init_state(stdin=stdin)
```



```
# Ensuring the first byte of the symbolic variable corresponds to 'a'
state.add_constraints(sym_variable.get_byte(0) == ord('a'))

# Creating a simulation manager to manage the execution of the state
simgr = proj.factory.simgr(state)

# Defining addresses to find and to avoid during symbolic execution
avoid_addr = [0x400c06, 0x400bc7]
find_addr = 0x400c10d

# Finding a state that satisfies the find condition while avoiding undesired states
simgr.explore(find=find_addr, avoid=avoid_addr)

# Printing the result if a satisfying state is found
if simgr.found:
    print(simgr.found[0].solver.eval(sym_variable, cast_to=bytes))
```



CYBER
CHALLENGE.IT



Signals

Ptrace can be utilized by programs to detect and prevent debugging, making it challenging to bypass such protections.

One possible strategy is **patching the binary** by removing the ptrace call.

But in some cases, patching is not possible or sufficient. Here the solution is to log system functions by **redefining them**.

- A. Create a shared library where functions of interest are redefined to include logging
- B. Use **LD_PRELOAD** to overwrite the call



Ptrace Bypass

```
#include <stdio.h>
#include <sys/ptrace.h>

int main() {
    if (ptrace(PTRACE_TRACEME, 0, 0, 0) < 0) {
        printf("I'm being debugged!\n");
    } else {
        printf("Normal flow\n");
    }
    return 0;
}

long ptrace(int request, int pid, void *addr, void *data) {
    return 0;
}
```



gcc -shared ptrace.c -o ptrace.so



In GDB

export LD_PRELOAD=./ptrace.so



In the shell

set environment LD_PRELOAD=./ptrace.so



GDB allows to specify which **process** to follow when a fork occurs:

- To follow the parent process: `set follow-fork mode parent`
- To follow the child process: `set follow-fork mode child`

GDB can be configured to change the behavior of the debugger when a specific signal is triggered:

Syntax: `handle [SIGNAL] [ACTION]`

- To stop the program when a SIGTRAP signal is encountered:
`handle SIGTRAP stop`



Automated debugging techniques simplify the analysis of intricate routines

- **GDB initialization script:** gdb at startup executes the .gdbinit script, which can be customized with specific commands for automatic debugging
- **Python scripts:** create a python script containing all gdb commands to be executed automatically. Use the 'import gdb' directive.
- **pwntools integration:** pwntools can be used to attach to the target process and send gdb instructions



Command	Description
<code>file <executable></code>	Sets the specified executable as the current working file for debugging.
<code>b *<address></code>	Sets a breakpoint at the specified address in the program.
<code>run</code>	Starts the program and runs it until it hits a breakpoint.

Use the same commands as gdb



```
import gdb

# Pass a command to gdb to set the executable for debugging
gdb.execute("file ./binary_to_be_analysed")

# Set a breakpoint at a specific memory address
gdb.execute("b *" + hex(0x97))

# Run the program with an optional input parameter
gdb.execute("r " + optional_input_parameter)

# Step to the next instruction
gdb.execute("s")

# Assign the instruction at the current instruction pointer to a variable
s = gdb.execute("x/i $rip", to_string=True)[0]

# Set the RAX register to a specified value
gdb.execute("set $rax=" + value_to_set)
```

To make the script execute
`gdb -q -x your_gdb_script.py`



Utilize the [attach and debug](#) functions within your Python 3 script and set the parameter `'api=True'` to grant the script access to GDB's features.

```
from pwn import *

# Start the process to be debugged
io = process('executable_path')

# Attach GDB to the process, setting breakpoints and starting the execution
p = gdb.attach(io, gdbscript="""
br *main
r
""", api=True)

# Interact with the process: send data, execute GDB commands, and receive output
io.sendline('var_to_send')

# Execute GDB command and get the result as a string
data = io.gdb.execute("i r $rax", to_string=True) # Inspect the RAX register

# Receive a line from the process
a = io.recvline()

# Print the received output
print(a)
```

Same command as
the previous slide



Command	Description
<code>from pwn import *</code>	Imports the pwntools library, useful for binary exploitation.
<code>io = process('executable_path')</code>	Starts the process 'executable_path' and returns an object to interact with it.
<code>p = gdb.attach(io, gdbscript='' br*main r '')</code>	Attaches GDB to the process and runs a script to set a breakpoint at the start of 'main' and then runs the program.
<code>io.sendline(var_to_send)</code>	Sends a line to the process' standard input.
<code>a = io.recvline()</code>	Receives a line from the process's standard output.
<code>print(a)</code>	Prints the received line to the standard output.



- A debugging library for analyzing and manipulating binaries at runtime: <https://libdebug.org/>
- Helps in bypassing anti-debugging measures
- Can catch and handle signals
- Automatically detects and attaches to new threads
- Really helpful in extracting data from memory and registers at runtime
- NOT compatible with pwntools...



```
from libdebug import debugger

d = debugger("./test") # +

# Start debugging from the entry point
d.run() # +

my_breakpoint = d.breakpoint("function") # +

# Continue the execution until the breakpoint is hit
d.cont() # +

# Print RAX
print(f"RAX is {hex(d.regs.rax)}") # +
```



- A packer compresses, encrypts or obfuscates a binary (stub loader)
- How to recognize a packed binary:
 - Very small or very large sections
 - Unusual section names
 - High entropy (also with few imports)
- There are a lot of known packers: Themida, UPX, VMProtect...
- If the packer is custom, usually the best approach is a dynamic one





CYBER
CHALLENGE.IT

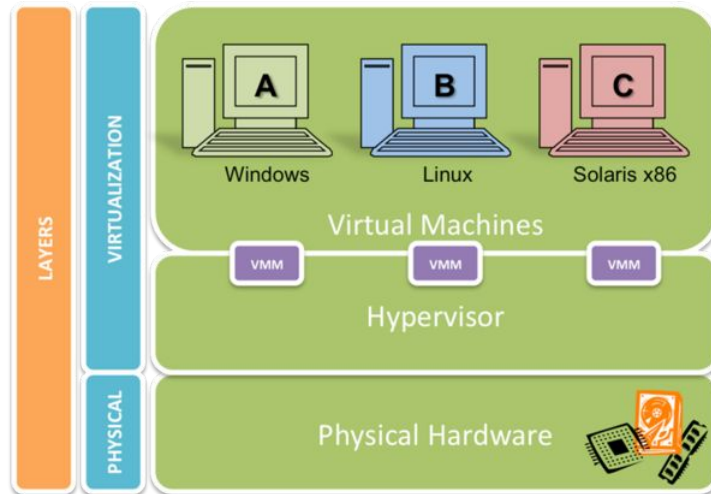


Reverse a Virtual Machine



A **Virtual Machine** is a software that virtualizes a different **Operating System** in your machine.

Given a machine with its own hardware and OS (*host machine*), you can virtualize the hardware and have many different other OSs (*guest machine*). This is possible thanks to the **hypervisor**.





Generally, in a VM challenge you have:

- an *interpreter* (which implements the VM)
- a *program* (with the inputs to the VM)

Goal: understand the input script according to the functioning of the interpreter

1. **In practice** (*low level*): write a disassembler to make the program more readable ☐ Study the interpreter.
2. **Logically** (*high level*): think of it as a physical machine ☐ given some input bytes (hardcoded, stdin, file), interpret the instructions and execute them on the program memory
3. **Statically:** follow next steps.
4. **Dynamically:** run the VM



- **Understand instructions:** CISC/RISC, number of bytes (if CISC). If you are not able to segment the instructions, do not disassemble
- **Find loops:** you have an offset counter and a switch for each opcode. How many instructions are there in the switch case (instruction set)? Which part of the instruction is the opcode and which one the instruction parameters?
- **Single instructions:** first understand every element of the emulated architecture



- **Volatile memory:** find the registers and understand the memory address, load/store mechanism
- **Stack structure:** LIFO or as a real machine with memory mapping. Understand this part from the instructions and how they interact with the memory.
- **Struct:** VM runs in sandbox mode and uses classes/structs to manage the interpreter. You need to write structs to help the decompiler have a more meaningful code.



Disassembler takes as input the program and produces as output a readable text with a sequence of instructions.

- **Linear:** simpler and easy to implement. Converts every byte to instruction by scanning the offset, without following branches. Cannot convert non-aligned code (jump at offset not-coherent with previous section) or data sections.
- **Recursive:** starts from an entry-point (usually address of the file), not emulate the program behaviour but follows the execution flow, by following jump and branch instructions. Accurate implementation. Easy to be transformed as emulator/debugger.



CYBER
CHALLENGE.IT



Challenges



This challenge is very special, can you find the better way to solve it?
We think that this challenge can be easily solved.

Flag format: wrap the found message in `srdnlen{}`





I can't find the correct input in order to make the program print "SUCCESS".
Can you find it for me?

Flag format: you will find a recognizable flag not in srdnlen format. To get the points in our platform, unwrap the flag and put it in srdnlen.

smth{this_is_your_flag} becomes srdnlen{this_is_you_flag}





I got this in my inbox, but I really can't understand what's inside...

Maybe you can help me in...

Flag format: you will find a recognizable flag not in srdnlen format. To get the points in our platform, unwrap the flag and put it in srdnlen.

smth{this_is_your_flag} becomes srdnlen{this_is_you_flag}





We got a weird binary and static analysis got us nowhere.
Can you find out what is dynamically happening?

Flag format: you will find a recognizable flag not in srdnlen format. To get the points in our platform, unwrap the flag and put it in srdnlen.

smth{this_is_your_flag} becomes srdnlen{this_is_your_flag}





What a funny VM!

Can you write your own disassembler to solve it?

Flag format: you will find a recognizable flag not in srdnlen format. To get the points in our platform, unwrap the flag and put it in srdnlen.

smth{this_is_your_flag} becomes srdnlen{this_is_you_flag}





You are lost in the forest, can you get special signals to go out?

Be safe and read the flag

Flag format: you will find a recognizable flag not in srdnlen format. To get the points in our platform, unwrap the flag and put it in srdnlen.

smth{this_is_your_flag} becomes srdnlen{this_is_you_flag}

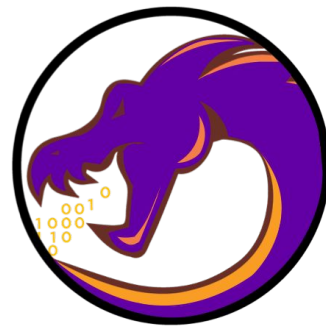




Crackers won't pirate our software anymore. Our serial number checks are really exceptional!

<https://ctf.cyberchallenge.it/challenges#challenge-202>

Flag format: submit CCIT flag also in local platform





Can you manage to beat a game without using your eyesight?

Flag format: wrap flag in `srdnlen{`





CYBER
CHALLENGE.IT



Extra Material



Use Ghidra extension to write python code to manage the reversing part.
Go *Window -> Python* to open a Python interpreter

Importance:

- Save data from decompiled files
- Run bash programs interacting with *analyzeHeadless* script in support directory

Example: extract from strings and function list (even choosing the xrefs cd)



```
#print strings
from ghidra.program.util import DefinedDataIterator
for s in DefinedDataIterator.definedStrings(currentProgram):
    print(s)

#find functions
fm = currentProgram.getFunctionManager()
funcs = fm.getFunctions(True)
print(funcs)
for func in funcs:
    a = func.getName()
    a = a.encode('ascii', 'ignore')
    print(a)

#find xrefs (after having found functions as above)
refs = currentProgram.referenceManager.getReferencesTo(toAddr(a))
xrefs = [i for i in refs]
```



- IDA plugin to execute python code
- Use for patching (patch a DRM)
- https://www.hex-rays.com/products/ida/support/idadpython_docs/
- <https://github.com/idadpython/src/tree/master/examples>



- Dynamic instrumentation toolkit for developers, reverse-engineers, security researchers: <https://frida.re/>
- Free and available for every OS (desktop and mobile)
- Default tools built on top of Frida API (use as-is or as examples)
- Inject your own scripts into black box processes
- Hook and trace functions from command-line, call native process functions
- Spy on crypto APIs
- Trace private application code
- Python API but JavaScript debugging logic



Frida – Why use?

INSTALLATION

Install	<code><i>pip install frida-tools</i></code>
Use	<code><i>frida-trace -i 'recv*' PROCESS_NAME</i></code>

Use cases

Interacting iOS apps based on encrypted protocols (Wireshark is not enough)	Frida track API
Desktop application with logging problems (create expensive customised version)	Frida to build a specific tool with all the necessary diagnostics (a few lines of Python)
Sniffing encrypted protocols, manipulating function calls to simulate network conditions	Frida to intercept and manipulate function calls without building a test lab
Create black-box test cases without modifying the application code	Frida interacts with the app to intercept specific conditions



Frida is the command with which you can interact with the process through terminal

<code>frida-ps</code>	Listing processes
<code>frida-trace</code>	Dynamically tracing function calls
<code>frida-discover</code>	Discovering internal functions in a program (traced next)
<code>frida-ls-devices</code>	Listing attached devices
<code>frida-kill</code>	Killing processes
<code>gum-graft</code>	Patching binaries to allow Interceptor when the binary cannot be modified at runtime



- Spawn an existing program or attach to a running program or hijack an already spawned program and run some script on it
- Frida executes Javascript, either directly through the **frida** command or through the python scripting
- Frida employs interceptors to inject code automatically
- Once the process is intercepted, frida
- Enumerate installed apps, running processes, connected devices (iOS and Android where running *frida-server*)



```
import frida
import sys
script = """
function intercept(address) {
    Interceptor.attach(address, {
        onEnter: function(args) {
            pass #instead of pass, write the actions performed by the code
        },
        onLeave: function(retval) {
            pass
        }
    });
}

intercept(0x300000);
"""

# Define message handler
def on_message(message, data):
    print(message)

# Example script to attach to a process by name
process_name = "..."
pid = frida.spawn(process_name) # spawns a new process
session = frida.attach(pid) # attaches to process
script = session.create_script(script)
script.on('message', on_message)
script.load()
frida.resume(pid)
sys.stdin.read()
```



- Spawn an existing program or attach to a running program or hijack an already spawned program and run some script on it
- *frida-core*: logistic layer to inject into existing software, providing two-way communication channel.
- Enumerate installed apps, running processes, connected devices (iOS and Android where running *frida-server*)



- Follow threads
- Capture every function, block, instruction executed
- Implementation architecture specific (aarch64 common Android – iOS, Intel 64 and IA-32 common on desktop and laptops)
- Invoked using Gum Interface or JavaScript API
- *Example: `Stalker.follow([threadId, options])`*



- E9tool is useful for static instrumentation
 - Binary Rewriter
- [e9patch/doc/e9tool-user-guide.md at master · GJDuck/e9patch \(github.com\)](https://github.com/GJDuck/e9patch/blob/master/doc/e9tool-user-guide.md)
- Use: `e9tool -M LOGIC -P ACTION`
 - e.g., **`e9tool -M jmp -P print bin`** creates a binary file from bin that prints all jmp instructions
 - You can apply regular expressions to it